

석사논문연구 중간 보고서

자동차공학과 석사과정 23512093 곽제우

1. Introduction

2019-2021년 캘리포니아의 AV 충돌 분석 결과, AV는 일반 차량보다 4.8배 더 많은 후방 추돌을 당했으며, 전체 충돌의 51.9%가 후방 충돌이었다. 특히 AV가 자율주행 모드일 때 후방 충돌 위험이 18.7% 더 높았고, 이는 주로 AV가 다른 도로 사용자에게 양보하거나 정지할 때 발생했다. 이러한 문제를 해결하기 위해서는 후방 차량을 더 잘 감지하고 처리하는 AV 성능 개선이 필요하다. 고속도로 추돌사고는 매우 심각한 교통안전 문제이다. 2023년 한국의 경우 고속도로 교통사고 중 후방 추돌이 약 40%를 차지하며 가장 높은 비중을 보였고, 사망사고의 경우 후방 추돌이 전체의 약 30%를 차지했다. 특히 화물차량이 관련된 후방 추돌 사고는 치명적인 결과로 이어지는 경우가 많았다.

미국의 경우 NHTSA(미국도로교통안전국) 자료를 보면, 전체 교통사고의 약 28%가 후방 추돌 사고였으며, 고속도로에서 발생하는 후방추돌의 약 87%가 운전자 부주의로 인한 것으로 나타났다. 후방추돌은 충격을 예측하기 어려워 대비가 불가능하고, 고속 주행 중 발생하므로 충격이 매우 크다. 연쇄 추돌로 이어질 경우에는 위험성이 더욱 증가한다. 초기 2중 추돌에 비해 사망률이 약 3-4배 증가하며, 부상자 수도 기하급수적으로 늘어난다. 최초 사고 이후 발생하는 2차 추돌은 더 큰 피해를 야기하며, 정지된 차량들로 인한 시야 제한으로 후속 차량의 회피가 어렵다. 이러한 사고를 예방하기 위해서는 충분한 차간거리 유지와 전방 주시가 매우 중요하다.

본 연구는 기존 AV 차량의 후방 추돌 안전 기능을 개선한 새로운 후방 접근 차량 추돌 방지 알고리즘을 제시한다. 특히 인명사고와 연쇄 추돌의 위험이 높은 화물차량의 추돌 사고를 효과적으로 방지하는 데 중점을 두었다. 이 알고리즘을 통해 AV의 고속도로 주행 시 발생할 수 있는 차량 추돌사고를 사전에 예방할 수 있을 것으로 기대된다.

2. Theory

① DQN

Q-Learning은 환경과의 상호작용을 통해 최적의 행동 정책을 학습한다. Q-learning의 핵심은 상태(s)에서 특정 행동(a)을 했을 때 얻을 수 있는 미래의 보상 기대값을 나타내는 Q-함수에 있다. Q-함수는 즉각적인 보상과 감가율을 고려한 다음 상태에서의 최대 Q값을 통해 계산되며, 이를 통해 각 상태에서 어떤 행동이 최선인지 판단할 수 있다. 학습 과정에서 에이전트는 환경과 상호작용하며 경험을 쌓고, 매 스텝마다 Q-테이블을 업데이트한다. 이때 ϵ -greedy 정책을 사용하여 탐험과 활용의 균형을 맞추며, 시행착오를 통해 점진적으로 최적의 행동 정책을 학습한다.

Q-learning은 모델에 대한 사전 지식 없이도 학습이 가능하고, Off-policy 학습이 가능하여 탐험에 유리하며, 수렴성이 증명되어 있다는 장점이 있다. 하지만 상태-행동 공간이 큰 경우 테이블로 표

현하기 어렵고, 연속적인 상태나 행동을 다루기 어려우며, 학습 속도가 느릴 수 있다는 단점도 있다. 이러한 한계를 극복하기 위해 DQN과 같은 발전된 알고리즘들이 제안되었다.

DQN(Deep Q-Network)은 Q-learning의 한계를 극복하기 위해 딥러닝을 결합한 알고리즘이다. 기존 Q-learning이 테이블 형태로 Q값을 저장했던 것과 달리, DQN은 신경망을 통해 Q-함수를 근사한다. 이를 통해 높은 차원의 상태 공간도 효과적으로 처리할 수 있게 되었으며, 특히 이미지와 같은 고차원 입력도 직접 처리할 수 있게 되었다. DQN의 주요 특징인 Experience Replay는 에이전트가 경험한 데이터를 메모리에 저장하고 무작위로 샘플링하여 학습에 활용하는 방식이다. 이는 데이터 간의 상관관계를 줄이고 학습의 안정성을 높이는데 기여한다. 또 다른 핵심 요소인 Target Network는 네트워크를 메인 네트워크와 타겟 네트워크로 분리하여 운영하는 것이다. 타겟 네트워크는 일정 주기로 메인 네트워크의 파라미터를 복사 받아 업데이트되며, 이 역시 학습의 안정성을 향상시키는데 중요한 역할을 한다. 하지만 연속적인 행동 공간을 다루기 어렵다는 점과 초기 학습이 불안정할 수 있다는 한계점도 존재한다. 이러한 한계를 보완하기 위해 Double DQN, Dueling DQN, Rainbow DQN 등 다양한 개선 알고리즘들이 제안되었다.

② LSTM

LSTM(Long Short-Term Memory)은 장기 의존성 문제를 해결하기 위해 제안된 특별한 종류의 RNN(Recurrent Neural Network) 구조이다. 시계열 데이터나 순차적 데이터 처리에 매우 효과적인 LSTM의 핵심 구조는 셀 상태(Cell State)와 세 개의 게이트로 구성된다.

망각 게이트(Forget Gate)는 이전 정보 중 어떤 것을 버릴지 결정하며, 시그모이드 함수를 통해 0과 1 사이의 값을 출력하여 정보의 보존 정도를 결정한다. 입력 게이트(Input Gate)는 새로운 정보 중 어떤 것을 저장할지 결정하는데, 시그모이드 함수와 tanh 함수를 조합하여 새로운 정보의 중요도를 판단한다. 출력 게이트(Output Gate)는 업데이트된 셀 상태의 어떤 부분을 출력으로 내놓을지 결정하고, 이 필터링된 정보를 다음 시점으로 전달한다.

LSTM은 긴 시퀀스의 정보를 효과적으로 학습할 수 있고, 기울기 소실/폭발 문제를 완화하며, 선택적 정보 저장과 삭제로 효율적인 메모리 관리가 가능하다는 장점을 가진다. 이러한 특성으로 인해 자연어 처리, 음성 인식, 시계열 예측, 기계 번역, 이미지 캡셔닝 등 다양한 분야에서 널리 활용되고 있으며, 시퀀스 데이터 처리의 기본이 되는 알고리즘이다.

③ PER

PER(Prioritized Experience Replay)는 DQN의 Experience Replay 메커니즘을 개선한 방법으로, 2015년에 제안되었다. 기존의 Experience Replay가 모든 경험을 동일한 확률로 샘플링했던 것과 달리, PER은 더 중요한 경험을 더 자주 샘플링하여 학습 효율성을 높이는 방식이다.

PER의 핵심은 우선순위 할당과 샘플링 확률 계산, 그리고 편향 보정이다. 우선순위는 TD 오차의 크기를 기반으로 부여되며, 예상치 못한 전환에 더 높은 우선순위가 할당된다. 우선 순위 경험 재

생(Prioritized Experience Replay, PER)은 훈련에 있어서 어떤 경험이 다른 것보다 더 중요할 수 있지만, 덜 자주 발생할 수 있다는 것이다.

Replay buffer에서 배치를 균등하게 샘플링할 때(경험을 무작위로 선택할 때) 이러한 드물게 발생하는 중요한 경험들은 실제로 선택될 기회가 거의 없다. 왜냐하면 episode 하나에 1000~2000 step(10~20초)의 데이터가 쌓이고 15~30가지 random한 상황의 episode를 Replay buffer에 넣고 학습하기 위해서 사이즈를 30,000으로 설정했고, batch size는 64이므로 1 step에서 학습하는 데이터는 전체 Replay buffer의 0.064%이다. 그렇기 때문에 PER을 사용하여 각 Replay buffer에서 학습할 데이터의 우선 순위를 정의하는 기준을 사용하여 샘플링 분포를 변경해 학습하면 학습이 필요한 데이터가 우선적으로 학습이 될 것이다. 예측과 TD 목표 사이에 차이가 큰 데이터에 우선순위를 주어 학습을 진행한다. TD 목표에서 많이 벗어난 예측을 한 데이터가 쌓였기 때문에 이때의 상황을 더 개선해야 하기 때문에 더 학습이 필요한 것이다. 그러나 탐욕적인 우선순위 지정은 항상 동일한 경험을 훈련시켜 과적합하게 하기 때문에 그냥 이용할 수 없다. 그래서 그냥 이용할 수 없다. 그래서 리플레이를 위한 선택될 확률을 생성하는 확률론적 우선순위를 도입한다.

$$P(i) = \frac{p_i^a}{\sum_k p_k^a} \quad (0.1)$$

a 는 Replay buffer에 대한 경험 선택에서 일부 임의성을 다시 도입하는데 사용되는 하이퍼파라미터이다. 만약 a 가 0이면 완전 랜덤추출이고, a 가 1이면 가장 높은 우선순위만 선택한다. p_i^a 는 우선순위 값이고, $\sum_k p_k^a$ 로 나누어 Replay buffer의 모든 우선 순위 값으로 정규화 한다.

결과적으로, 각 시간 단계마다 이 확률 분포에 따라 샘플의 배치를 얻고, 이를 바탕으로 네트워크를 훈련한다. 하지만 일반적인 경험 재생(Experience Replay)을 사용할 때는 확률적인 업데이트 규칙을 따르며, 따라서 경험을 샘플하는 방식은 그 경험이 유래한 원래의 분포를 반영해야 한다. 일반적인 경험 재생에서는 경험을 정규 분포에서 무작위로 선택한다. 즉, 우선 순위가 낮은 경험보다 학습에 더 자주 사용될 가능성이 높기 때문이다. 그 결과, 관심 있는 경험의 작은 부분만으로도 가중치를 업데이트하게 될 것이다. 이 편향(bias)을 교정하기 위해 중요도 샘플링 가중치(Importance Sampling Weights)를 사용하여 자주 보이는 샘플의 가중치를 줄여 업데이트를 조정한다. 우선 순위가 높은 샘플에 해당하는 가중치는 거의 조정되지 않지만(네트워크가 이 경험을 자주 볼 수 있기 때문에) 우선 순위가 낮은 샘플에 해당하는 가중치는 더 크게 업데이트 된다.

$$\left(\frac{1}{N} \cdot \frac{1}{P(i)}\right)^\beta \quad (0.2)$$

β 는 학습에 미치는 영향을 제어한다. 학습 시작 시 0에 가깝고 학습 기간 동안 최대 1까지 변한다. 그 이유는 Q값이 수렴하기 시작할 때 학습의 끝에서 이러한 가중치가 더 중요하기 때문이다.

다. N 은 Replay Buffer의 크기이다. $P(i)$ 는 샘플링 확률이다.

④ N-step TD

N-step 학습은 TD(Temporal Difference) 학습과 Monte Carlo 학습의 중간적 접근 방식으로, n개의 타임스텝 동안의 보상을 누적하여 학습에 사용하는 방법이다. 기존의 1-step TD 학습이 즉각적인 보상과 다음 상태의 가치만을 고려했던 것에 비해, N-step은 n개의 스텝 동안의 실제 보상을 사용함으로써 더 정확한 가치 추정이 가능하다. TD 학습은 현재 상태의 추정 가치와 다음 상태의 추정 가치의 차이를 이용하여 학습하는 방법이다. TD 학습의 가장 큰 특징은 실제 에피소드가 끝날 때까지 기다리지 않고 매 단계마다 학습이 가능하다는 점이다. 이는 부트스트래핑 방식을 사용하여 다른 추정치를 기반으로 학습하기 때문이다. TD 학습의 기본적인 업데이트 규칙은 식 0.3으로 표현된다.

$$V(s_t) \leftarrow V(s_t) + \alpha \{R_{t+1} + \gamma V(s_{t+1}) - V(s_t)\} \quad (0.3)$$

여기서 $V(s_t)$ 는 현재 상태의 가치, α 는 학습률, R_{t+1} 은 즉각적인 보상, γ 는 할인율, $V(s_{t+1})$ 는 다음 상태의 가치를 나타낸다. 이러한 업데이트 규칙을 통해 각 상태의 가치를 점진적으로 개선해 나간다. TD 학습은 온라인 학습이 가능하고 낮은 분산을 가진 추정치를 생성할 수 있어 실제 환경에서 더 실용적이다. TD 학습의 대표적인 알고리즘으로는 on-policy TD 학습인 SARSA, off-policy TD 학습인 Q-learning, 그리고 여러 단계의 TD 예측을 결합한 TD(λ) 등이 있다. Monte Carlo 방법은 경험으로부터 직접적으로 학습하는 강화학습의 한 방법이다. 이는 에피소드를 완료한 후 실제로 얻은 보상을 통해 가치함수를 추정하는 방식이다. Monte Carlo 방법의 핵심은 실제 경험을 통해 얻은 리턴값의 평균을 계산하여 상태나 행동의 가치를 추정한다는 것이다.

Monte Carlo 방법의 학습 과정은 각 에피소드가 종료된 후, 에피소드에서 방문한 각 상태에 대해 관찰된 리턴값을 사용하여 가치함수를 갱신한다. 새로운 추정치는 이전 추정치와 실제 관찰된 리턴값 사이의 차이에 학습률을 곱한 값을 더하여 계산된다.

$$V(s) \leftarrow V(s) + \alpha (G_t - V(s)) \quad (0.4)$$

이는 식 0.4로 표현되며, 여기서 $V(s)$ 는 상태 s 의 가치, α 는 학습률, G_t 는 특정 시점 t 부터 에피소드 종료 시점까지 받은 보상들의 할인된 값의 합으로 식 0.5와 같다.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \dots + \gamma^{T-t-1} R_T \quad (0.5)$$

여기서 T 는 에피소드의 종료 시점, γ 는 할인율 ($0 \leq \gamma \leq 1$), R_t 는 t 시점의 보상을 나타낸다. 할인율 γ 는 미래의 보상이 현재에 비해 얼마나 중요한지를 결정하는 파라미터이다. γ 가 0에

가까울수록 즉각적인 보상을 중요하게 여기고, 1에 가까울수록 미래의 보상도 현재만큼 중요하게 고려한다. 예를 들어 $\gamma = 0.9$ 일 때, 10스텝 후의 보상은 현재 가치의 약 35%만큼의 가치를 가진다.

Monte Carlo 방법은 환경에 대한 사전 지식이 필요하지 않고 실제 경험만으로 학습할 수 있다는 장점이 있다. 또한 TD 학습과 달리 부트스트래핑을 사용하지 않기 때문에 초기 가치 추정의 편향에 영향을 받지 않는다. 그러나 에피소드가 완료될 때까지 기다려야 하므로 온라인 학습이 불가능하고, 리턴값의 분산이 크다는 단점이 있다. Monte Carlo 방법은 First-visit Monte Carlo와 Every-visit Monte Carlo로 나뉘며, 각각 에피소드 내에서 상태의 첫 방문만을 고려하거나 모든 방문을 고려한다는 차이가 있다.

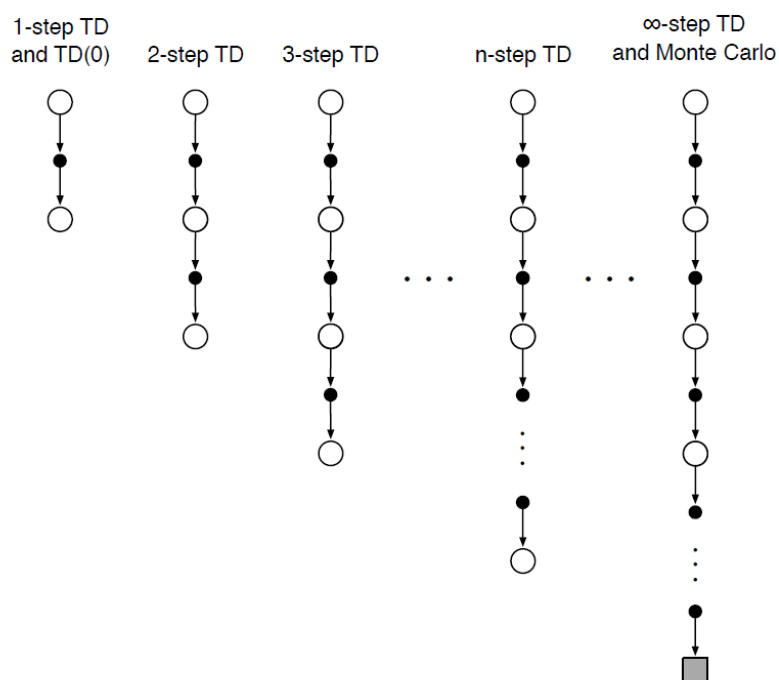


Figure 1 N-step TD

N-step의 핵심은 n개의 스텝 동안 발생하는 보상을 모두 고려하여 현재 상태의 가치를 업데이트 한다는 점이다. 이는 TD 오차를 계산할 때 n개의 스텝 동안의 실제 보상과 n번째 스텝 이후의 예측 가치를 조합하여 사용한다. Fig.1에 N-step TD에서 나타난 것처럼 n이 1이면 일반적인 TD 학습이 되고, n이 에피소드의 길이와 같아지면 Monte Carlo 방법이 된다. 따라서 n값의 선택은 편향(bias)과 분산(variance) 사이의 트레이드오프를 조절하는 중요한 하이퍼파라미터가 된다.

N-step 학습은 더 정확한 가치 추정과 향상된 학습 효율성이라는 장점이 있지만, 구현이 복잡해지고 메모리 사용량이 증가하며, 적절한 n값 선택이 필요하다는 단점도 있다. 이 방법은 Rainbow DQN을 포함한 여러 현대적인 강화학습 알고리즘에서 중요한 구성 요소로 사용되고 있으며, 특히 긴 시간 의존성을 가진 문제에서 효과적인 성능을 보여준다.

3. RL Framework

① Object Oriented Program

객체 지향 프로그래밍(OOP)의 장점은 코드의 구조화와 재사용성 측면에서 매우 두드러진다. 현실 세계의 개념을 프로그래밍으로 자연스럽게 표현할 수 있어 직관적인 설계가 가능하며, 이는 개발자가 복잡한 시스템을 더 쉽게 이해하고 관리할 수 있게 해준다. 또한 캡슐화를 통해 데이터와 해당 데이터를 처리하는 메서드를 하나의 단위로 묶어 관리함으로써, 코드의 안정성이 높아지고 유지보수가 용이 해진다. 상속 기능을 활용하면 기존 클래스의 기능을 새로운 클래스에서 재사용할 수 있어 코드 중복을 줄일 수 있으며, 이는 개발 시간을 단축시키고 버그 발생 가능성을 낮춘다. 다형성을 통해 같은 인터페이스로 다양한 객체를 다룰 수 있어 코드의 유연성이 증가하며, 이는 시스템의 확장성을 높여준다. 특히 대규모 프로젝트에서는 이러한 OOP의 특성들이 코드의 구조화와 관리를 더욱 효율적으로 만들어주어, 여러 개발자가 협업하는 환경에서도 일관성 있는 코드 작성이 가능해진다.

② Sumo_RL Framework

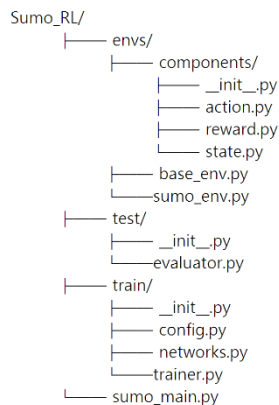


Figure 2 Sumo RL Structure

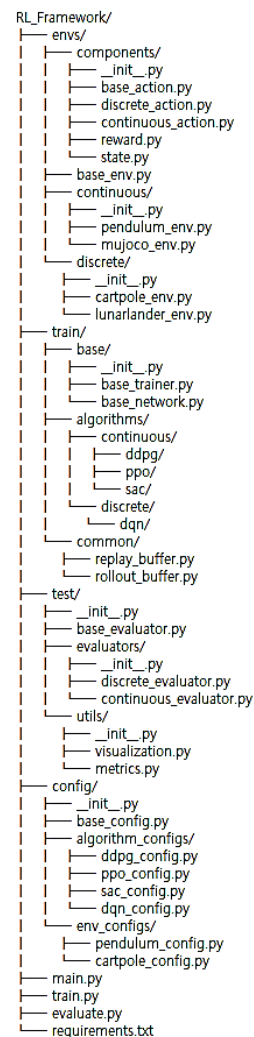


Figure 3 RL Framework (Sumo RL extension)

코드 구조 Fig.2와 같이 구성되며, 내부 구조는 class로 만들었다. envs, test, train를 각각 Class로 캡슐화 하여 구현했고, 이것의 장점은 환경과 알고리즘이 추가하기 쉽고 따라서 Fig.3의 구조처럼 DDPG, PPO, SAC과 같은 딥강화학습 알고리즘을 추가해서 관리함으로써 환경 통합관리와 성능비교를 용이하게 할 수 있다.

4. Method: Anti-Rear approaching vehicle collision algorithm(후방 접근 차량 충돌 방지 알고리즘)

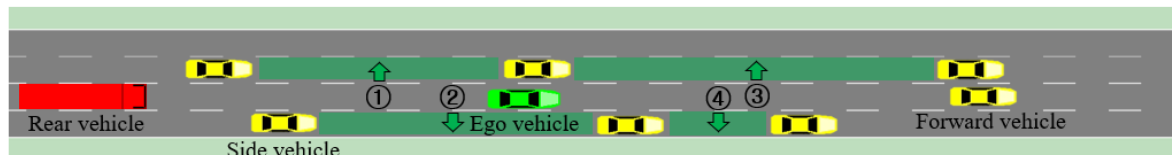
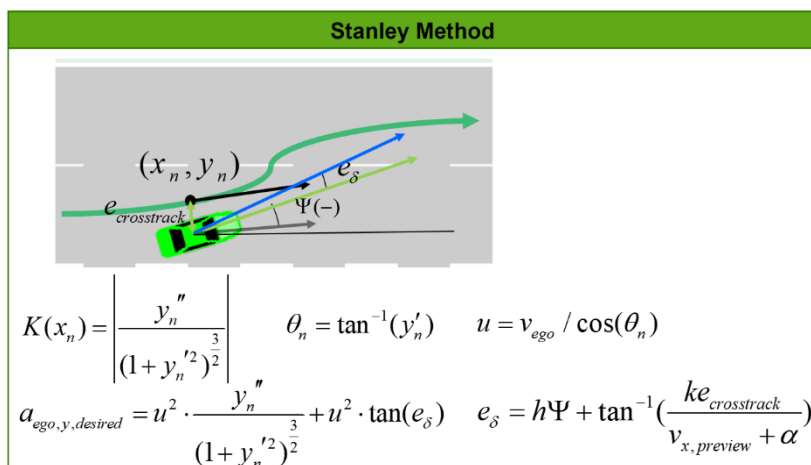
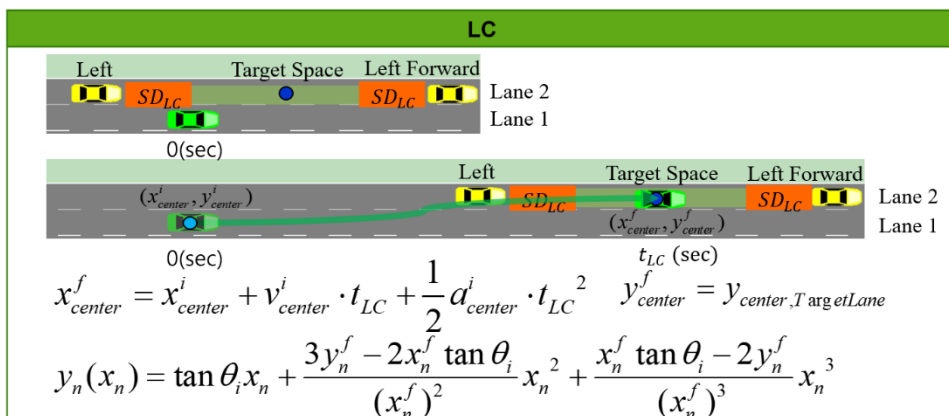


Figure 4 Anti-Rear approaching vehicle episode.

후방 접근 차량 충돌 방지 알고리즘이란 후방의 부주의한 화물차량의 이상 거동을 탐지했을 때, 차로 변경을 통해서 회피주행을 하는 알고리즘이다. 자차(ego vehicle)은 주변 8대의 차량을 관심 차량으로 하여 상대 거리, 상대속도 등의 상태를 기반으로 회피 주행을 판단하고, Fig. 4에 표시된 4개의 공간 중 하나로 차로 변경한다. 차로 변경을 위해서 실시간 경로 계획을 수행하기 위해서 다이나믹 경로를 3차 다항식 경로를 생성[1] 하고, 경로 추종을 위해서 Stanley method[2]를 사용했다.



① Rule Based Algorithm(규칙 기반 알고리즘)

규칙 기반 알고리즘은 후방차량의 추돌을 예측하면 옆 차로 공간 중 가장 가까운 공간으로 이동하는 알고리즘이다. 차로 변경이 성공할 때까지 가까운 공간 탐색은 계속 이뤄진다.

② DRL Based Algorithm(딥강화학습 기반 알고리즘)

딥강화학습 기반 알고리즘은 DQN의 변형을 사용했다. Google DeepMind에서 제안한 최초의 DQN[3]은 4step의 state를 받아서 사용하는 반면 본 연구에서 제안하는 방식은 100step의 state 정보를 누적해서 LSTM을 거쳐서 나온 hidden state를 DQN의 입력으로 사용한다. DQN에 LSTM 추가해서 주변 차량의 거동을 시계열로 입력 받으면 시간에 대해서 한 차원 더 높게 표현할 수 있다. 뿐만 아니라 [4]에서 제안한 PER과 n-step TD 알고리즘을 적용해서 학습 성능을 높였다. PER을 적용한 이유는 학습이 필요한 데이터를 우선적으로 학습하도록 하기 위해서이다. 그리고 5-step TD 알고리즘을 적용하여 5초 동안의 보상을 직접 고려하도록 했다. 즉, 차선 변경 시작, 차선 변경 중, 차선 변경 완료의 일련의 주행 동작 중 연속된 동작에 대한 보상을 고려할 수 있다. 이는 식 0.5에서 나타난 것처럼 t 시점의 Return은 t 시점 뿐만 아니라 t+1, t+2, ..., t+n-1 시점의 보상을 누적 합하여 학습에 고려하는 것을 의미한다.

Fig.5는 PER과 5-step TD 알고리즘의 흐름을 보여준다. 5 step의 transition(state, action, reward, next state, done)을 순차적으로 deque 자료 구조에 저장한다. 5개의 transition이 모이면 n-step 리턴을 계산하는데, 이는 각 단계의 reward에 감가율(discount factor)을 적용하여 합산하는 방식이다. 예를 들어, 5-step에 대한 리턴은 $R = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \gamma^4 r_5$ 와 같이 계산한다. 그 다음 첫 번째 상태(S_t)의 Q값과 마지막 상태(S_{t+5})의 Q값을 사용하여 TD error를 계산한다. 이렇게 계산된 TD error는 priority로 변환하여 PER(Prioritized Experience Replay)에 경험과 함께 저장한다. Buffer는 새로운 transition이 들어올 때마다 FIFO(First In First Out) 방식으로 가장 오래된 transition을 제거하고 새로운 데이터를 추가하여, 항상 최근 5개의 transition만을 유지한다. 이러한 n-step TD buffer 구조는 여러 단계에 걸친 보상을 직접적으로 고려할 수 있게 해주어, 에이전트가 장기적인 결과를 더 효과적으로 학습할 수 있도록 도와준다.

후방 접근 차량 추돌 방지 알고리즘을 학습하기 위해서 Reward는 Positive reward와 Negative reward로 구분해서 설계했다. Positive reward는 차로 변경을 위한 타깃 공간까지의 거리가 작을수록 큰 보상을 받고 선택한 공간의 크기가 클수록 큰 보상을 갖는다. 그리고 차로 변경에 성공할 경우 보상을 받는다. Negative Reward의 경우 자차가 다른 차량과 충돌이 발생할 경우 그리고 주행 중 전방 및 주변 차량과의 안전거리를 침범했을 경우 음의 보상을 갖는다.

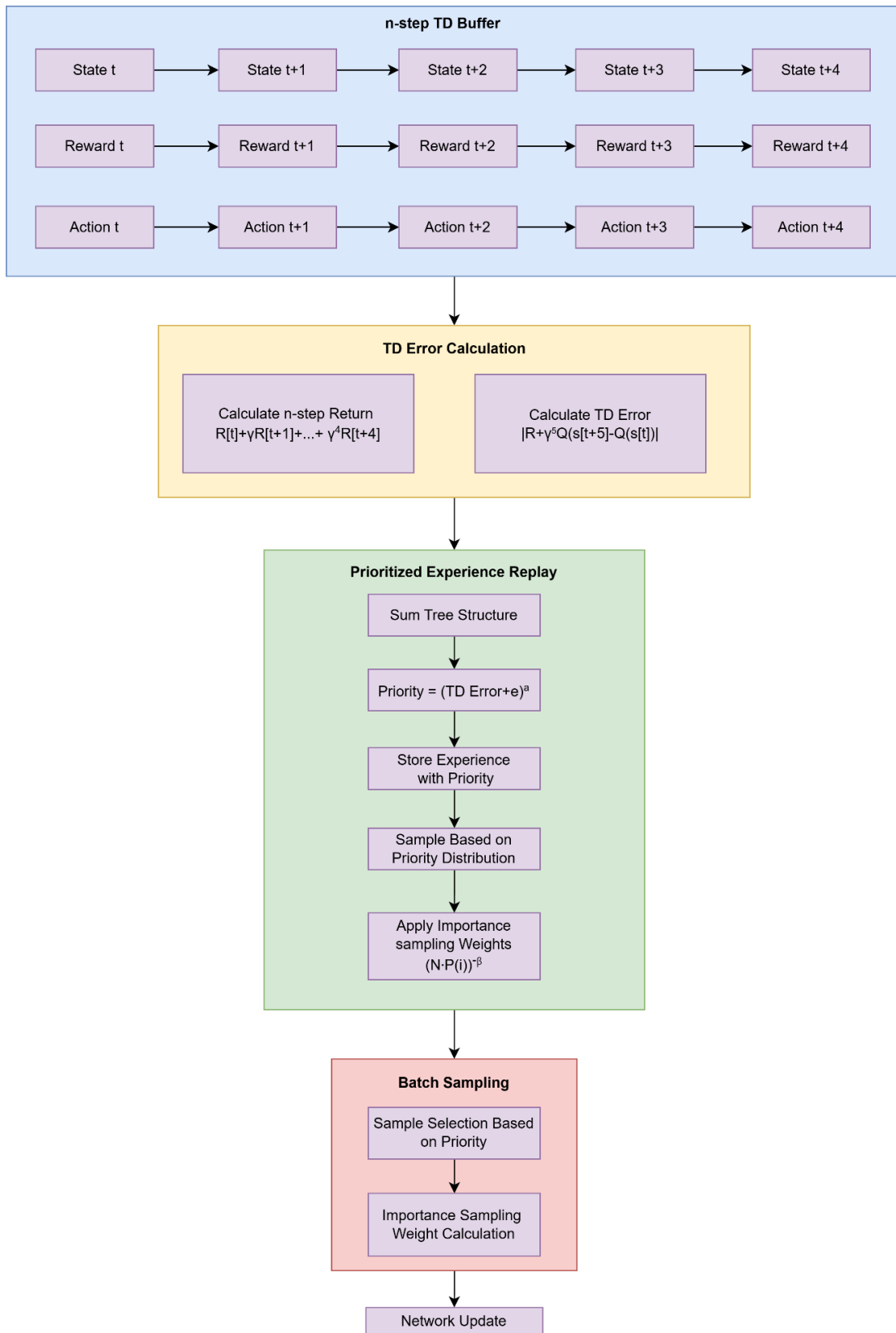


Figure 5 PER and 5-step TD Flow Chart

n-step DQN은 Fig.6에서 나타난 방식으로 네트워크를 업데이트한다. 앞서 Fig.5에서 Replay buffer에 저장한 데이터를 PER 알고리즘에 따라 Q-Network에 입력되고, Q-Network가 학습한 결과로 Q-value가 출력되어 현재 step의 reward와 Target-Network의 Q-value의 최대값의 감쇠계수를 곱한 값과 합의 차이를 Loss(MSE Loss)로 해서 학습을 진행한다. Target Network는 Q-Network의 네트워크 파라미터를 1 episode 간격으로 복제한다. 이는 Target network에서 예측한 Q-value를 사용하는 방식이기 때문에 Bootstrapping(예측값을 이용한 계산) 방식이다.

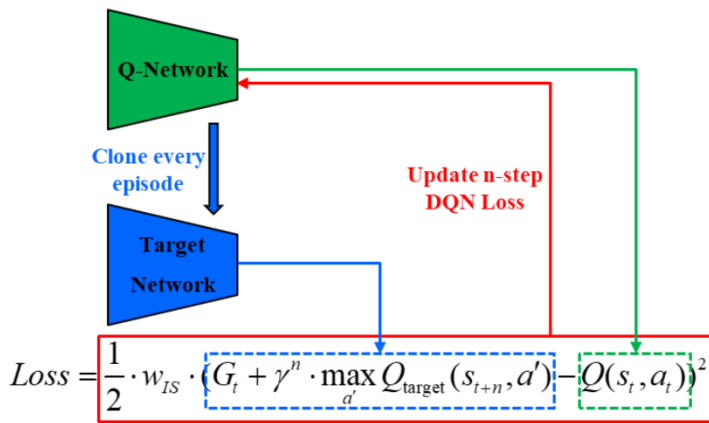


Figure 6 Network Update

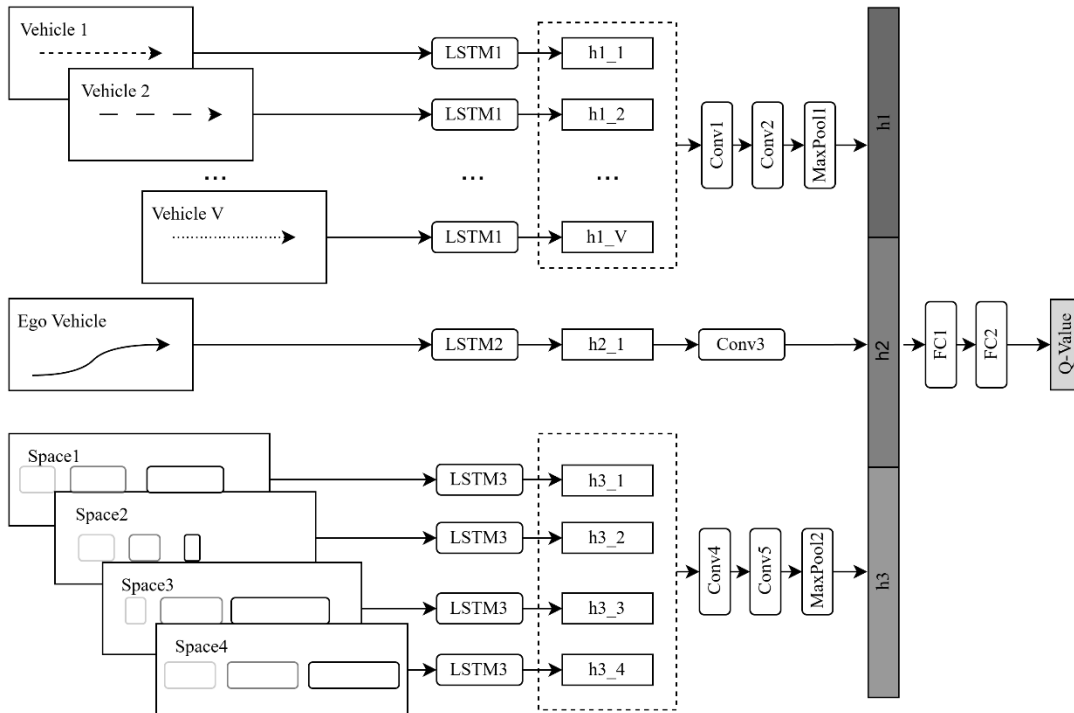


Figure 7 Network

Q-Network와 Fig.7에 표현된 구조를 갖고, Target network도 이와 동일한 구조를 갖는다. 그렇기 때문에 1 episode 마다 Q-Network의 파라미터를 복제한 것을 Target network에서도 그대로 사용

할 수 있는 것이다. Network 구조는 다음과 같다. 우선 Fig.8에 나타난 것처럼 주변의 8대 차량의 상태 값과, 자차의 상태 값, 그리고 옆차로의 4개의 공간에 대한 상태 값을 받아서 LSTM을 거쳐서 hidden state으로 함축한 후 concatenate하여 차원을 일치시킨 뒤 1D convolution layer를 거친다. 이것은 주변 차량, 자차, 차로 변경 공간의 시계열 값을 1D 공간 이미지로 표현해서 학습하는 방식이다. 이 구조는 1차원의 데이터로 네트워크를 학습하기 때문에 입력을 차량 주변의 Bird's eye view 2차원 이미지처럼 고차원의 데이터로 학습하지 않아서 학습 속도와 계산 속도에 이점을 갖는다.

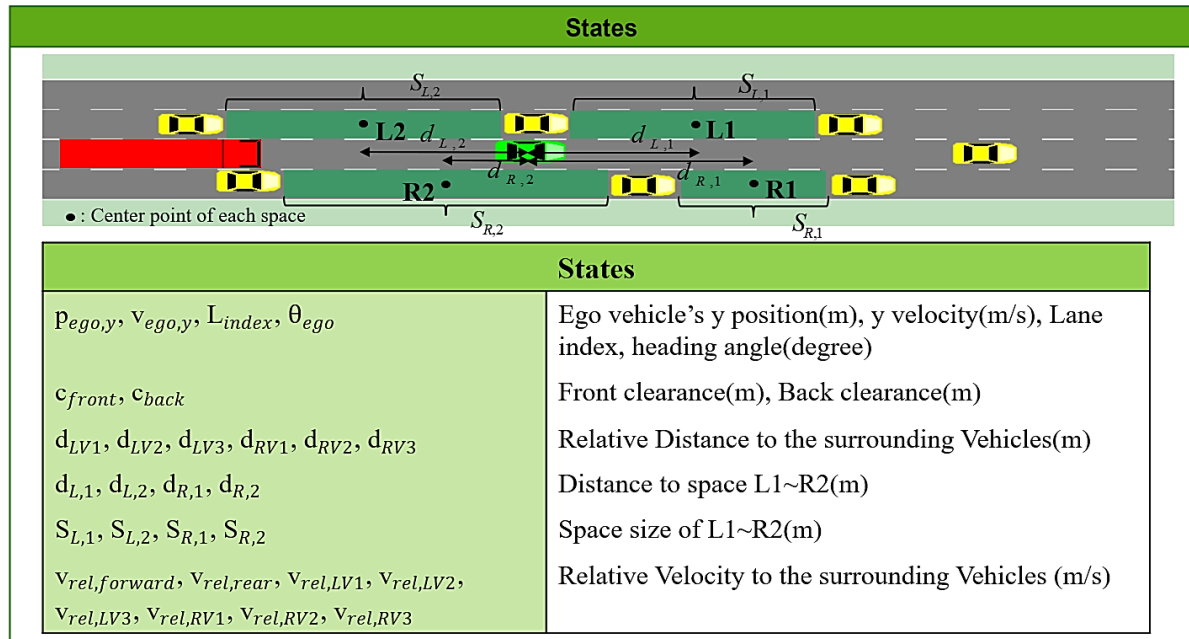


Figure 8 State

5. Results

규칙 기반 알고리즘과 딥강화학습 알고리즘을 통해서 후방 접근 차량 회피 주행 성능을 비교한 결과는 Table 1에서 확인할 수 있다. Table 1은 차로 변경 성공 확률을 나타낸 표이다. 본 연구에서 제안한 딥강화학습 구조(LSTM+DQN+PER+n-step TD)가 가장 높은 확률을 갖는다.

	Rule based	LSTM+DQN	LSTM+DQN+PER+n-step TD
LC success proportion	71.6%	50.1%	86.3%

Table 1 Lane change success probabilities of rule-based and DRL algorithms.

다음의 Fig.8~9는 강화학습 성능을 나타낸 그래프이다. (a)는 누적 충돌 횟수, (b)는 보상의 이동 평균, (c)는 차로 변경에 성공했을 때 그 공간의 크기에 대한 누적 순위, 그리고 (d)는 차로 변경에 성공했을 때 공간의 누적 수를 그래프로 표현한 것이다. Fig.8에서 나타난 LSTM+DQN의 누적 충돌 횟수를 봤을 때 충돌이 점차 감소하지만 약 50%의 확률로 충돌이 발생하는 것을 확인할 수 있다. 보상의 이동 평균 그래프에서는 보상이 점차 증가함을 알 수 있다. 그리고 보상함수에서 의도한 결과처럼 큰 공간으로 차로 변경을 가장 많이 하는 것을 확인할 수 있었고, L1(왼쪽 전방)의 공간으로 차로 변경의

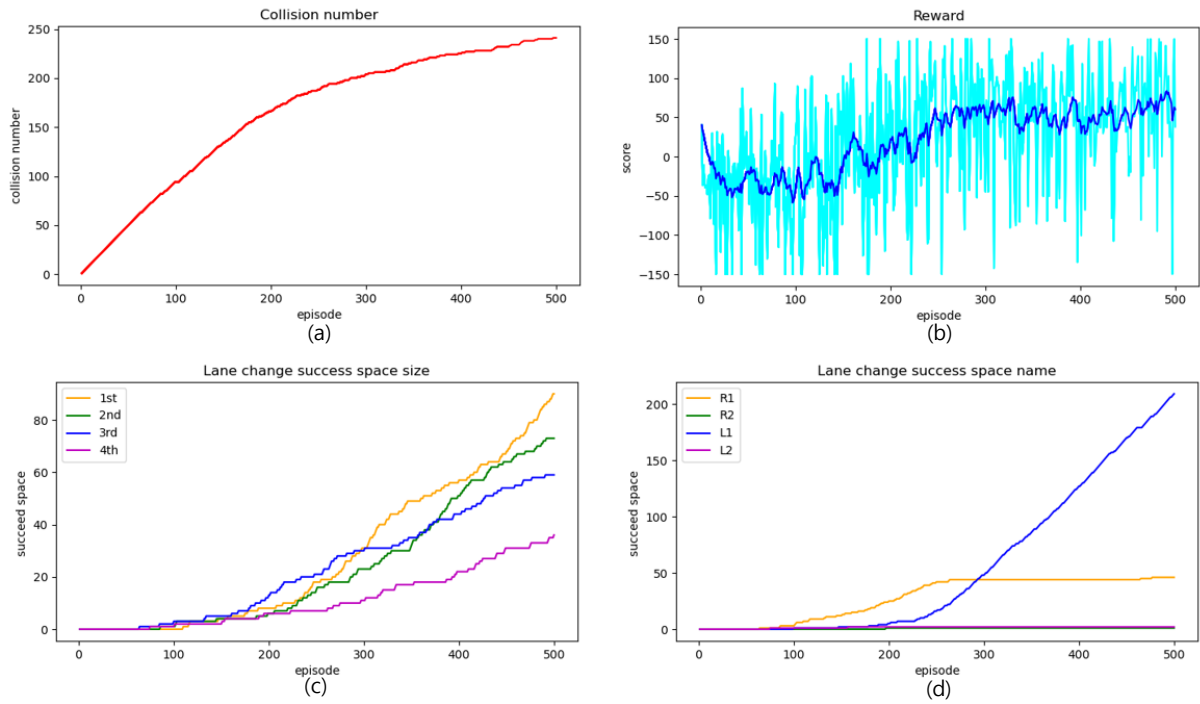


Figure 9 LSTM DQN Result.

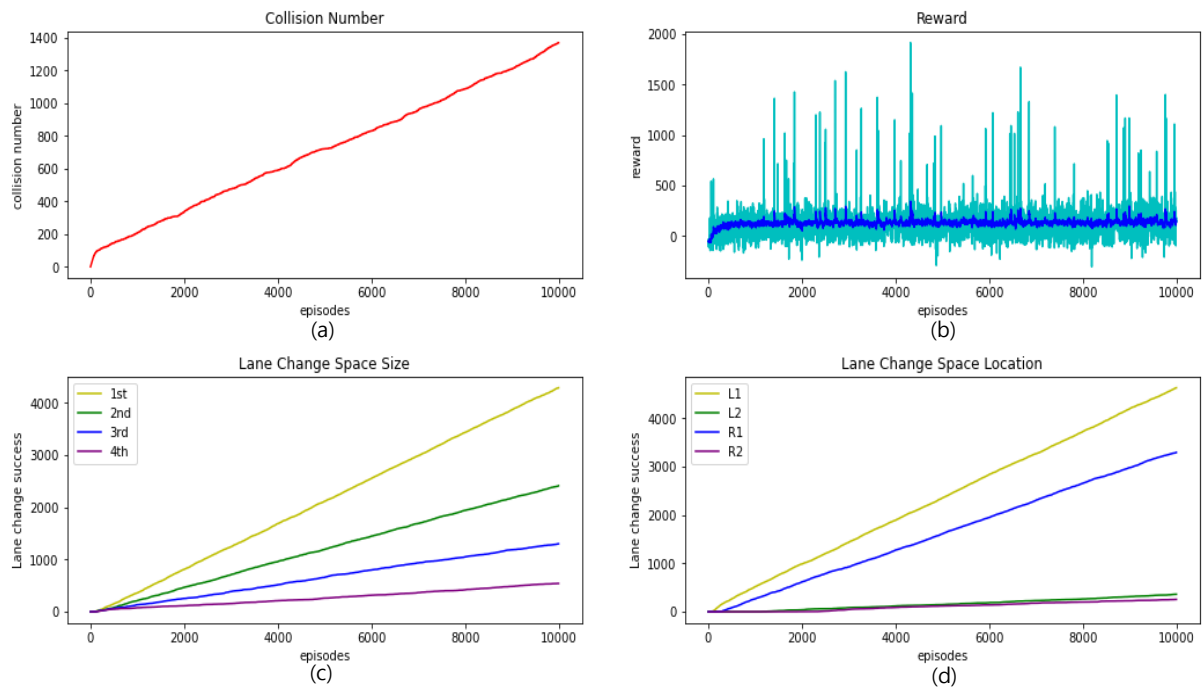


Figure 10 LSTM DQN PER n-step TD Result.

빈도가 가장 많은 것을 확인할 수 있다. 한 공간으로 차로 변경을 지속하는 경향을 봤을 때 오버 피팅에 가능성 존재한다.

다음으로 Fig.9의 (a)를 확인했을 때 충돌이 일정 수준을 유지하는 것을 볼 수 있다. 이는 후방 차량의 접근 속도에 비해서 빠른 판단이 이뤄지지 않은 경우에 충돌이 발생했기 때문이다. (b)에서 reward는 일정 학습초기 1000 episodes까지 이동 평균 보상이 증가하는 것은 학습이 잘 진행되고 있는 것을 알 수 있고, 이후부터는 정책이 수렴해서 큰 증가를 보이지 않는 것으로 보인다. (c)에서 확인할 수 있듯이 가장 큰 공간으로 차로 변경을 대부분 수행하며, (d)에서 확인할 수 있듯이 학습한 정책은 L1, R1(전방 좌우 공간)으로 차로 변경을 대부분 수행하는 것을 알 수 있다. 이는 대부분의 episodes에서

후방 차량을 피하기 위해서 전방의 공간으로 차로 변경하는 것이 최적으로 계산한 것으로 보이며, 일부의 경우 L2, R2(후방 좌우 공간)으로 차로 변경하는 것은 선행차량과 너무 근접해 있을 때 후방의 좌우 공간을 선택해서 차로 변경을 수행한 것으로 확인했다.

6. Discussion

본 연구에서 제안한 DQN에 LSTM를 추가해서 PER 알고리즘과 n-step TD 알고리즘으로 86.3% 확률로 후방 접근 차량 추돌 회피를 성공했다. 다음의 최신 딥강화학습 알고리즘을 적용해서 추돌 회피 성능을 높일 수 있을 것으로 기대할 수 있다.

① PPO

PPO[55]는 이산 행동과 연속 행동을 출력할 수 있고 Atari게임과 MuJoCo(humanoid 등) 예시에서 성능이 검증되었다. 따라서 기존에 구축한 SUMO 환경을 이용해서 바로 테스트할 수 있다.

PPO는 Actor network와 Critic network 2개를 한 번에 학습시키는 방법으로 on-policy로 정책을 업데이트하면서 안정적으로 Actor-critic을 학습할 수 있는 방법이다. PPO는 구현이 쉽고 Clipping을 통해 policy update의 크기를 제한하였고 Trust region 제약을 통해 학습이 안정적이고 좋은 성능을 보인다는 점이 큰 강점이다.

② DDPG (Deep Deterministic Policy Gradient)

DDPG는 continuous action space에서 작동하는 off-policy actor-critic 알고리즘이다. 따라서 SUMO 환경을 연속 행동으로 변경해서 학습을 진행해야 한다. 이 알고리즘은 DQN의 아이디어를 continuous action에 적용했으며, deterministic policy를 직접 학습한다는 특징이 있다.

핵심 메커니즘은 actor와 critic 두 개의 네트워크를 사용하는 것이다. Actor는 현재 상태에서 최적의 action을 출력하는 정책을 학습하고, critic은 action-value function을 학습하여 actor가 출력한 action의 가치를 평가한다. 학습의 안정성을 위해 target network를 사용하며, experience replay를 통해 sample efficiency를 높인다.

DDPG의 주요 강점은 다음과 같다. 첫째, continuous action space에서 효율적으로 작동한다. 기존 DQN은 discrete action만 다룰 수 있었지만, DDPG는 연속적인 제어가 필요한 중,형 속도 제어 문제에 직접 적용할 수 있다. 둘째, off-policy 학습이 가능하여 sample efficiency가 높다. 이전에 수집된 데이터를 재사용할 수 있어 실제 환경과의 상호작용을 줄일 수 있다. 셋째, deterministic policy를 직접 학습하기 때문에 policy gradient의 high variance 문제를 완화할 수 있다.

단, DDPG는 학습이 불안정할 수 있고 하이퍼파라미터에 민감하다는 단점이 있다. 이를 보완하기 위해 TD3나 SAC 같은 발전된 알고리즘들이 제안되었다.

③ SAC (Soft Actor-Critic)

SAC은 maximum entropy 강화학습 프레임워크를 기반으로 하는 off-policy actor-critic 알고리즘이다. 이 알고리즘은 reward 최대화와 동시에 정책의 엔트로피를 높이는 것을 목표로 한다.

핵심 메커니즘은 두 가지 Q-function과 하나의 policy network를 사용하는 것이다. 두 개의 Q-

function은 overestimation bias를 줄이기 위해 사용되며, policy network는 확률적 정책을 학습한다. 또한 자동으로 entropy 계수를 조절하는 temperature parameter를 도입하여 exploration과 exploitation의 균형을 맞춘다.

SAC의 주요 강점은 다음과 같다. 첫째, 높은 sample efficiency를 가진다. Off-policy 학습과 entropy maximization을 통해 효율적인 학습이 가능하다. 둘째, 안정적인 학습이 가능하다. Double Q-learning과 자동 entropy 조절 메커니즘으로 인해 학습이 안정적이다. 셋째, exploration과 exploitation의 균형이 좋다. 정책의 엔트로피를 높임으로써 다양한 행동을 탐색하면서도 좋은 행동을 활용할 수 있다.

SAC는 DDPG나 TD3와 달리 확률적 정책을 사용하여 더 robust한 학습이 가능하지만, 계산 복잡도가 높다는 단점이 있다. 그럼에도 현재 continuous control 문제에서 가장 널리 사용되는 알고리즘이다.

7. Reference

- [1] Y. Da, Z. Shiyu, W. Cheng, J. Peter, and R. Bin, "A dynamic lane-changing trajectory planning model for automated vehicles," *Transportation Research Part C: Emerging Technologies*, vol. 95, pp. 228-247, 2018, doi: <https://doi.org/10.1016/j.trc.2018.06.007>
- [2] S. Jarrod, "Automatic steering methods for autonomous automobile path tracking," *Robotics Institute, Pittsburgh, PA, Tech. Rep. CMU-RITR-09-08*, 2009.
- [3] M. Volodymyr et al., "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529-533, 2015.
- [4] H. Matteo et al., "Rainbow: Combining improvements in deep reinforcement learning," in *Proceedings of the AAAI conference on artificial intelligence*, 2018,.
- [5] S. John, W. Filip, D. Prafulla, R. Alec, and K. Oleg, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.